- **跨域**
  - 同源策略
    - 编程中的同源，比较的是网页所在url和Ajax请求url是否同源。
    - 主要看下面三个方面：
      - 协议是否相同（http https file）
      - 主机地址是否相同（只看前面的地址或IP，后面的/xxx/xxx/xxx.....没影响）
      - 端口（0~65535）（http默认端口是80；https默认端口是443；MySQL默认端口3306）
    - 协议、主机地址、端口组成一个“源”。
    - 网页所在url的协议、域名、端口号，和Ajax请求url的协议、域名,、端口号都相同就是同源，有一个不一样就是跨域。
    - 如果非同源，那么以下三种行为会受到限制：
      - 浏览器存储 无法操作（实测这句话对Cookies不对。Cookies只根据hostname区分，如果hostname相同，就算protocol、port不同，依然共用同一个Cookies）
      - DOM 无法获得
      - Ajax请求结果被浏览器拦截（请求可以发送，服务器也会处理这次请求，但是响应结果会被浏览器拦截）
    - 违反了同源策略的请求，叫做跨域请求。
  - 可能产生跨域的场景
    - 由 XMLHttpRequest 或 Fetch API 发起的跨源 HTTP 请求
    - Web 字体（CSS 中通过 @font-face 使用跨源字体资源），因此，网站就可以发布 TrueType 字体资源，并只允许已授权网站进行跨站调用
    - WebGL 贴图
    - 使用 drawImage() 将图片或视频画面绘制到 canvas
    - 来自图像的 CSS 图形 (en-US)
  - 解决跨域
    - 主流的方案有两种：分别是JSONP和CORS.
    - JSONP
      - JSONP方案和Ajax没有任何关系
      - JSONP方案只支持GET请求
      - JSONP没有浏览器兼容问题，任何浏览器都支持。
      - 原理
        - 客户端利用 script 标签的 src 属性，去请求一个接口，因为src属性不受跨域影响。
        - 服务端响应一个字符串
        - 客户端接收到字符串，然后把它当做JS代码运行。
      - 后端接口代码：

```
app.get('/api/jsonp', (req, res) => {
  // res.send('hello');
```

```
  // res.send('console.log(1234)');
  // res.send('abc()')
  // res.send('abc(12345)')

  // 接收客户端的函数名
  let fn = req.query.callback;
  let obj = { status: 0, message: '登录成功' };
  let str = JSON.stringify(obj);
  res.send(fn + `(${str})`);
});
```

- 前端代码：

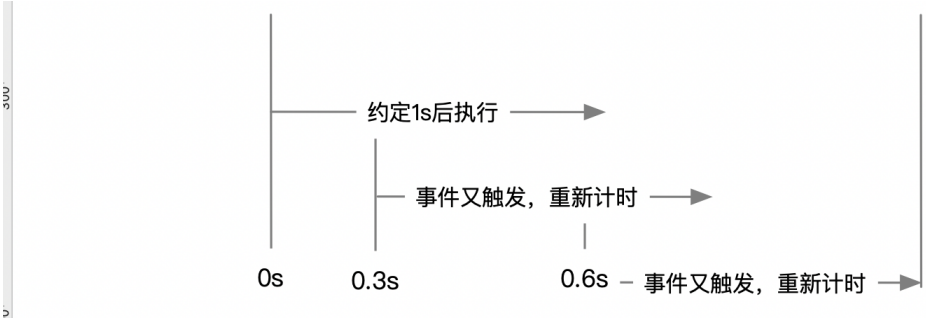
```
<script>
  // 提前准备好一个函数
  function xxx(res) {
    console.log(res)
  }

</script>

<script src="http://localhost:3006/api/jsonp?callback=xxx"></script>
```

- 前端需要做什么？
  - 如果使用jQuery，$.ajax({ dataType: 'jsonp' })，必须指定dataType选项为jsonp即可
- 后端需要做什么？
  - 如果使用express，那么直接调用 `res.jsonp(数据)` 即可。

- CORS
  - 由于XHR对象被W3C标准化之后，提出了很多XHR Level2的新构想，其中新增了很多新方法（onload、response....）和CORS跨域资源共享。浏览器升级后开始支持CORS方案，从IE10开始支持。
  - CORS方案，就是通过服务器设置响应头来实现跨域。
  - CORS才是解决跨域的真正解决方案。
  - 前端需要做什么？
    - 无需做任何事情，正常发送Ajax请求即可。
  - 后端需要做什么？
    - 需要加[响应头](https://developer.mozilla.org/zh-CN/docs/Glossary/CORS) 。或者使用第三方模块 cors 。
  - 小结

方案	前端	后端
CORS	×	设置响应头
JSONP（原生）	1. 准备一个函数；2. 使用script的src发送请求	响应函数调用
JSONP（jQuery）	1. 还是调用$.ajax()；2. 必须指定dataType: 'jsonp'	res.jsonp(数据)

- **防抖和节流**
  - 防抖
    - 当事件触发之后，约定单位时间（比如1s）之后，执行里面的代码；如果在单位时间只内再次触发了事件，那么要`重新计时`，以保证事件里面的代码只执行一次。

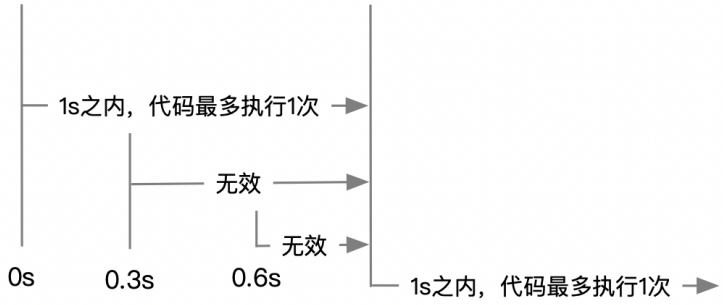



```
<style>
  * {
    margin: 0;
    padding: 0;
  }
  #box {
    width: 500px;
    margin: 20px auto;
  }
  ul,
```

```
11    li {
12      list-style: none;
13    }
14    input {
15      width: 100%;
16      height: 26px;
17      line-height: 26px;
18    }
19    li:hover {
20      background-color: beige;
21    }
22    ul {
23      display: none;
24    }
25  </style>
26  <div id="box">
27    <input type="text" id="ipt">
28    <ul></ul>
29  </div>
30  <script src="./jquery.js"></script>
31  <script>
32    let timer = null;
33    // 当输入框的键盘弹起的时候，发送请求，获取搜索建议
34    $('#ipt').on('keyup', function () {
35      // 清除前面的定时器
36      clearTimeout(timer);
37      // 获取输入的值（搜索关键字）
38      let keywords = $(this).val();
39      if (keywords === '') {
40        return $('ul').empty().hide();
41      }
42      // 如果关键字不为空，则获取搜索建议
43      // 约定 1s 之后发送请求
44      timer = setTimeout(() => {
45        $.ajax({
46          url: 'https://suggest.taobao.com/sug',
47          data: { q: keywords, code: 'utf-8' }, // 加入code参数，能够搜索多个汉字
48          dataType: 'jsonp', // JSONP请求必须加这项
49          success: function (res) {
50            // console.log(res)
51            let arr = [];
52            res.result.forEach(item => {
```

```
            arr.push(`<li>${result[0]}</li>`)
          })
          $('ul').html(arr.join('')).show();
        }
      });
    }, 1000);
  })
</script>
```

- 节流
  - 当事件触发之后，约定单位时间之内，事件里面的`代码最多只能执行 1 次`。
  - 所以，节流减少了单位时间内代码的执行次数，从而提高性能。



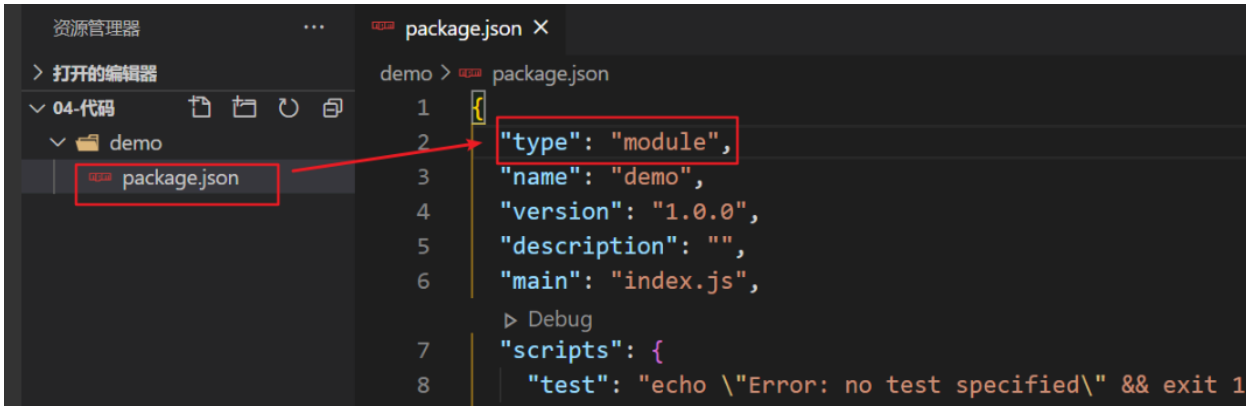
  - 使用 timer 当做开关（节流阀）。
    - 开关 打开状态（timer = null），则允许执行代码。
    - 开关是关闭状态（timer = 数字），则不允许执行代码。
  - 代码：

```
<style>
  html,
  body {
    height: 100%;
  }
  img {
    position: absolute;
```

```
  }
</style>
<img src="./angel.gif" alt="">
<script src="./jquery.js"></script>
<script>
  let timer = null; // null，表示节流阀打开状态，允许执行事件里面的代码
  let img = $('img');
  $(document).on('mousemove', function (e) {
    console.log(111)
    // 当事件触发了，判断一下，节流阀的状态，如果是关闭状态，则不允许创建另一个定时器
    if (timer !== null) return;
    timer = setTimeout(() => {
      console.log(222);
      let x = e.pageX;
      let y = e.pageY;
      // 设置图片的css（left和top）
      img.css({ left: x + 'px', top: y + 'px' });
      // 当定时器执行完毕，重新打开节流阀
      timer = null;
    }, 16);
  });
</script>
```

- **ES6模块化**
  - 模块化的好处
    - node.js 遵循了 CommonJS 的模块化规范。其中：
      - 导入其它模块使用 `require()` 方法
      - 模块对外共享成员使用`module.exports` 对象
    - 模块化的好处：
      - 模块化可以避免命名冲突的问题
      - 大家都遵守同样的模块化规范写代码，降低了沟通的成本，极大方便了各个模块之间的相互调用
      - 只需关心当前模块本身的功能开发，需要其他模块的支持时，在模块内调用目标模块即可
  - 模块化的分类
    - 在 ES6 模块化规范诞生之前，JavaScript 社区已经尝试并提出了`AMD`、`CMD`、`CommonJS` 等模块化规范。
    - 但是，这些由社区提出的模块化标准，还是存在一定的差异性与局限性、并不是浏览器与服务器通用的模块化标准，例如：
      - `AMD` 和 `CMD` 适用于浏览器端的 Javascript 模块化
      - `CommonJS` 适用于服务器端的 Javascript 模块化

        - 太多的模块化规范给开发者增加了学习的难度与开发的成本。因此，官方的ES6 模块化规范诞生了！
    - ES6 模块化规范
        - ES6 模块化规范是浏览器端与服务器端通用的模块化开发规范。它的出现极大的降低了前端开发者的模块化学习成本，开发者不需再额外学习 AMD、CMD 或 CommonJS 等模块化规范。
        - ES6 模块化规范中定义：
            - 每个 js 文件都是一个独立的模块
            - 导入其它模块成员使用 `import` 关键字
            - 向外共享模块成员使用 `export` 关键字
        - 在nodejs中使用ES6模块化
            - node.js 中默认仅支持 CommonJS 模块化规范，若想基于 node.js 体验与学习 ES6 的模块化语法，可以按照如下两个步骤进行配置：
                - 确保安装了 `v13.0.0` 或更高版本的 node.js
                - 在 package.json 的根节点中添加 `"type": "module"` 节点



- **ES6模块语法**
    - ES6 的模块化主要包含如下 3 种用法：
        - 默认导出与默认导入
        - 按需导出与按需导入
        - 直接导入并执行模块中的代码
    - 默认导出与默认导入
        - 默认导出的语法：`export default 默认导出的成员`
        - 默认导入的语法：`import 接收名称 from '模块路径'`
        - 导出

```
const a = 10
const b = 20

const fn  = () => {
  console.log('这是一个函数')
}

```

```
// 默认导出
// export default a  // 导出一个值
export default {
  a,
  b,
  fn
}
```

- 导入

```
// 默认导入时的接收名称可以任意名称，只要是合法的成员名称即可
import result from './xxx.js'
console.log(result)
```

- 注意点:
  - 每个模块中，只允许使用唯一的一次 `export default `
  - 默认导入时的接收名称可以任意名称，只要是合法的成员名称即可

- 按需导入与按需导出
  - 按需导出的语法： `export const s1 = 10`
  - 按需导入的语法： `import { 按需导入的名称 } from '模块标识符'`

```
export const a = 10
export const b = 20
export const fn = () => {
  console.log('内容')
}
```

- 按需导入的语法

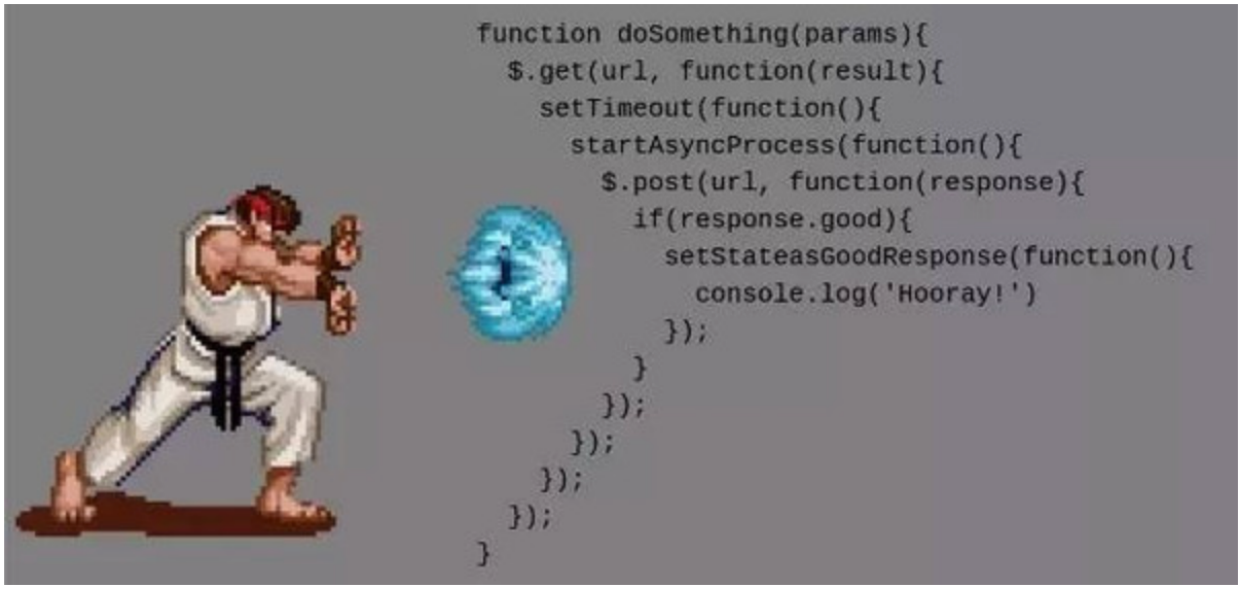
```
import { a, b as c, fn } from './xxx.js'
```

- 注意事项:
  - 每个模块中可以有多次按需导出
  - 按需导入的成员名称必须和按需导出的名称保持一致
  - 按需导入时，可以使用 as 关键字进行重命名
  - 按需导入可以和默认导入一起使用

- 直接导入模块(无导出)
  - 如果只想单纯地执行某个模块中的代码，并不需要得到模块中向外共享的成员。
  - 此时，可以直接导入并执行模块代码，示例代码如下：

```
//xxx.js
for (let i = 0; i < 10; i++) {
  console.log(i)
}
// 导入该模块
import './xxx.js'
```

- **Promise**
  - 回调地狱



  - JS中或node中，都大量的使用了回调函数进行异步操作，而异步操作什么时候返回结果是不可控的，如果我们希望几个异步请求按照顺序来执行，那么就需要将这些异步操作嵌套起来，嵌套的层数特别多，就会形成回调地狱或者叫做横向金字塔。
  - 下面的案例就有回调地狱的意思：
    - 案例：有 a.txt、b.txt、c.txt 三个文件，使用fs模板按照顺序来读取里面的内容，代码：

```
// 将读取的a、b、c里面的内容，按照顺序输出
const fs = require('fs');

// 读取a文件
fs.readFile('./a.txt', 'utf-8', (err, data) => {
    if (err) throw err;
    console.log(data.length);
```

```
    // 读取b文件
    fs.readFile('./b.txt', 'utf-8', (err, data) => {
        if (err) throw err;
        console.log(data);
        // 读取c文件
        fs.readFile('./c.txt', 'utf-8', (err, data) => {
            if (err) throw err;
            console.log(data);
        });
    });
});
```

- Promise简介
  - Promise对象可以解决回调地狱的问题，同时提高执行效率
  - Promise 是异步编程的一种解决方案，比传统的解决方案（回调函数和事件）更合理和更强大
  - `Promise可以理解为一个容器，里面可以编写异步程序的代码`
  - 从语法上说，Promise 是一个对象，使用的使用需要 `new`
- Promise简单使用
  - Promise是“承诺”的意思，实例中，它里面的异步操作就相当于一个承诺，而承诺就会有两种结果，要么完成了承诺的内容，要么失败。
  - 所以，使用Promise，分为两大部分，首先是有一个承诺（异步操作），然后再兑现结果。
  - 第一部分：定义“承诺”

```
// 实例化一个Promise，表示定义一个容器，需要给它传递一个函数作为参数，而该函数又有两个形参，通常用resolve和reject来表示。该函数里面可以写异步请求的代码
// 换个角度，也可以理解为定下了一个承诺
let p = new Promise((resolve, reject) => {
    // 形参resolve，单词意思是 完成
    // 形参reject ，单词意思是 失败
    fs.readFile('./a.txt', 'utf-8', (err, data) => {
        if (err) {
            // 失败，就告诉别人，承诺失败了
            reject(err);
        } else {
            // 成功，就告诉别人，承诺实现了
            resolve(data.length);
        }
    });
});
```

- 第二部分：获取“承诺”的结果

```
// 通过调用 p 的then方法，可以获取到上述“承诺”的结果
// then方法有两个函数类型的参数，参数1表示承诺成功时调用的函数，参数2可选，表示承诺失败时执行的函数
p.then(
    (data) => {},
  (err) => {}
);
```

- 完整的代码：

```
const fs = require('fs');
// promise 承诺
// 使用Promise分为两大部分
// 1. 定义一个承诺
let p = new Promise((resolve, reject) => {
    // resolve -- 解决，完成了；是一个函数
    // reject  -- 拒绝，失败了；是一个函数
    // 异步操作的代码，它就是一个承诺
    fs.readFile('./a.txt', 'utf-8', (err, data) => {
        if (err) {
            reject(err);
        } else {
            resolve(data.length);
        }
    });
});

// 2. 兑现承诺
// p.then(
//     (data) => {}, // 函数类似的参数，用于获取承诺成功后的数据
//     (err) => {} // 函数类型的参数，用于或承诺失败后的错误信息
// );
p.then(
    (data) => {
        console.log(data);
    },
    (err) => {
```

```
        console.log(err);
    }
);
```

- 三种状态
  - 最初状态：pending，等待中，此时promise的结果为 undefined；
  - 当 resolve(value) 调用时，达到最终状态之一：fulfilled，（成功的）完成，此时可以获取结果value
  - 当 reject(error) 调用时，达到最终状态之一：rejected，失败，此时可以获取错误信息 error
  - 说明：当达到最终的 fulfilled 或 rejected 时，promise的状态就不会再改变了。
- 同步异步
  - new Promise是同步执行的
  - 获取结果时（调用 resolve 触发 then方法时）是异步的

```
console.log(1);

new Promise((resolve, reject) => {
  console.log(2);
  resolve();
  console.log(3);
}).then(res => {
  console.log(4);
})

console.log(5);

// 输出顺序： 1  2  3  5  4 ,因为只有 .then() 是异步的
```

- then方法的链式调用
  - 前一个then里面返回的字符串，会被下一个then方法接收到。但是没有意义；
  - 前一个then里面返回的Promise对象，并且调用resolve的时候传递了数据，数据会被下一个then接收到
  - 前一个then里面如果没有调用resolve，则后续的then不会接收到任何值

```
const fs = require('fs');
// promise 承诺

let p1 = new Promise((resolve, reject) => {
  fs.readFile('./a.txt', 'utf-8', (err, data) => {
    err ? reject(err) : resolve(data.length);
```

```
  });
});

let p2 = new Promise((resolve, reject) => {
  fs.readFile('./b.txt', 'utf-8', (err, data) => {
    err ? reject(err) : resolve(data.length);
  });
});

let p3 = new Promise((resolve, reject) => {
  fs.readFile('./c.txt', 'utf-8', (err, data) => {
    err ? reject(err) : resolve(data.length);
  });
});

p1.then(a => {
  console.log(a);
  return p2;
}).then(b => {
  console.log(b);
  return p3;
}).then(c => {
  console.log(c)
}).catch((err) => {
  console.log(err);
});
```

    - 说明：catch 方法可以统一获取错误信息
  - 封装按顺序异步读取文件的函数

```
function myReadFile(path) {
    return new Promise((resolve, reject) => {
        fs.readFile(path, 'utf-8', (err, data) => {
            err ? reject(err) : resolve(data.length);
        })
    });
}
myReadFile('./a.txt')
.then(a => {
  console.log(a);
```

```
  return myReadFile('./b.txt');
})
.then(b => {
  console.log(b);
  return myReadFile('./c.txt');
})
.then(c => {
  console.log(c)
})
.catch((err) => {
  console.log(err);
});
```

- 使用第三方模块读取文件
  - npm init -y
  - npm i then-fs　安装then-fs模块
  - then-fs 将 内置的fs模块封装了，读取文件后，返回 Promise 对象，省去了我们自己封装
  - 修改 package.json ，添加 "type": "module"　表示使用ES6的模块化语法

```
import fs from 'then-fs';

fs.readFile('./files/a.txt', 'utf-8')
  .then(res1 => {
    console.log(res1);
    return fs.readFile('./files/b.txt', 'utf-8')
  })
  .then(res2 => {
    console.log(res2);
      return fs.readFile('./files/b.txt', 'utf-8')
  })
    .then(res3 => {
    console.log(res3)
    })
```

- async 和 await 修饰符
  - async 和 await 是 ES2017 中提出来的。
  - 异步操作是 JavaScript 编程的麻烦事，麻烦到一直有人提出各种各样的方案，试图解决这个问题。
  - 从最早的回调函数，到 Promise 对象，再到 Generator 函数，每次都有所改进，但又让人觉得不彻底。它们都有额外的复杂性，都需要理解抽象的底层运行机制。

- 异步I/O不就是读取一个文件吗，干嘛要搞得这么复杂？异步编程的最高境界，就是根本不用关心它是不是异步。
- async 函数就是隧道尽头的亮光，很多人认为它是异步操作的终极解决方案。
- ES2017提供了async和await关键字。await和async关键词能够将异步请求的结果以返回值的方式返回给我们。
  - async 用于修饰一个 function
    - async 修饰的函数，总是返回一个 Promise 对象
    - 函数内的返回值，将自动包装在promise 的 resolved 或 rejected中
  - await 只能出现在 async 函数内
    - await 让 JS 引擎等待直到promise完成并返回结果
    - 语法：let value = await promise对象; // 要先等待promise对象执行完毕，才能得到结果
    - 由于await需要等待promise执行完毕，所以await会暂停函数的执行，但不会影响其他异步任务
  - 对于错误处理，可以选择在async函数后面使用 `.catch()` 或 在promise对象后使用 `.catch()`

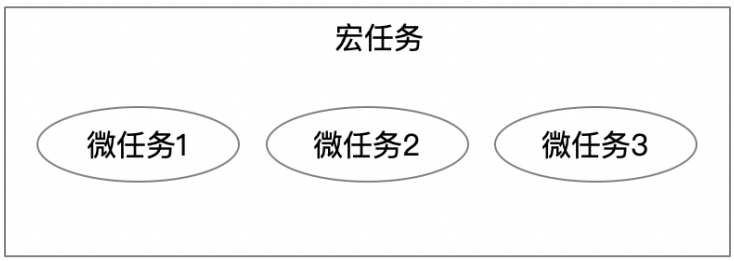
```
const fs = require('fs');
// 将异步读取文件的代码封装
function myReadFile (path) {
    return new Promise((resolve, reject) => {
        fs.readFile(path, 'utf-8', (err, data) => {
            err ? reject(err) : resolve(data.length);
        });
    }).catch(err => {
        console.log(err);
    });
}

async function abc () {
    let a = await myReadFile('./a.txt');
    let b = await myReadFile('./b.txt');
    let c = await myReadFile('./c.txt');
    console.log(b);
    console.log(a);
    console.log(c);
}

abc();
```

- **宏任务和微任务、事件循环**

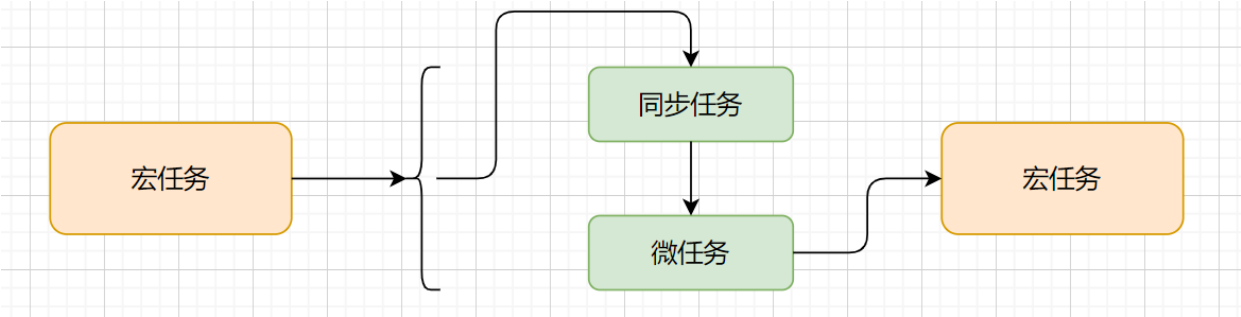
- 为什么JavaScript是单线程的
  - js是运行于浏览器的脚本语言，因其经常涉及操作dom，如果是多线程的，也就意味着，同一个时刻，能够执行多个任务。
  - 试想，如果一个线程修改dom，另一个线程删除dom，那么浏览器就不知道该先执行哪个操作。
  - 所以js执行的时候会按照一个任务一个任务来执行。
- 为什么任务要分为同步任务和异步任务
  - 试想一下，如果js的任务都是同步的，那么遇到定时器、网络请求等这类型需要延时执行的任务会发生什么?
  - 页面可能会瘫痪，需要暂停下来等待这些需要很长时间才能执行完毕的代码
  - 所以，又引入了异步任务。
    - 同步任务：同步任务不需要进行等待可立即看到执行结果，比如console
    - 异步任务：异步任务需要等待一定的时候才能看到结果，比如setTimeout、网络请求
- 宏任务和微任务
  - 异步任务，又可以细分为宏任务和微任务。下面列举目前学过的宏任务和微任务。

任务（代码）	宏/微 任务	环境
script	宏任务	浏览器
事件	宏任务	浏览器
网络请求（Ajax）	宏任务	浏览器
setTimeout() 定时器	宏任务	浏览器 / Node
fs.readFile() 读取文件	宏任务	Node
Promise.then()	**微任务**	浏览器 / Node

  - 他们的执行过程是怎样的呢?
  - 比如去银行排队办业务，每个人的业务就相当于是一个宏任务；
  - 比如一个人，办的业务有存钱、买纪念币、买理财产品、办信用卡，这些就叫做微任务。



- 执行顺序



- 事件循环
  - 事件循环比较简单，它是一个在 "JavaScript 引擎等待任务"，"执行任务"和"进入休眠状态等待更多任务"这几个状态之间转换的无限循环。
  - 引擎的一般算法：
    - 当有任务时：从最先进入的任务开始执行。
    - 休眠直到出现任务，然后转到第 1 步。

- **ES6降级处理**
  - 因为 ES 6 有浏览器兼容性问题，可以使用一些工具进行降级处理，例如：babel
  - 降级处理 babel 的使用步骤
    - 安装 Node.js
    - 命令行中安装 babel
    - 配置文件 `.babelrc`
    - 运行
  - 项目初始化 (项目文件夹不能有中文)

```
npm init -y
```

  - 在命令行中，安装 babel [babel官网](https://babeljs.io)

```
npm install  @babel/core @babel/cli @babel/preset-env
```

  - 配置文件 `.babelrc` (手工创建这个文件)
    - babel 的降级处理配置

```
{
  "presets": ["@babel/preset-env"]
}
```

  - 在命令行中，运行

```
# 把转换的结果输出到指定的文件
npx babel index.js -o test.js
# 把转换的结果输出到指定的目录
npx babel 包含有js的原目录 -d 转换后的新目录
```

- **身份认证**
  - 开发模式
    - 传统的服务端渲染模式
      - 后端的接口和前端的代码在一起（服务器）
      - 涉及不到跨域
      - 有利于SEO
      - 客户端（前端）不需要渲染数据，如果是手机，将会非常省电。运行速度非常快。
      - 缺点是开发效率低。
      - 适合使用cookie或session身份认证
    - 新型的前后端分离模式
      - 前端代码单独在一个文件夹（服务器）（自己的电脑上）
      - 后端的接口在另外的文件夹（服务器）（刘龙宾老师的服务器上）
      - 开发速度快，适合多人协作开发。
      - 适合使用JWT（json web token）方式的身份认证。
  - 演示传统的服务端渲染模式
    - 最大的特点
    - 服务端代码和前端代码在同一个服务器（文件夹）
    - 搭建服务器

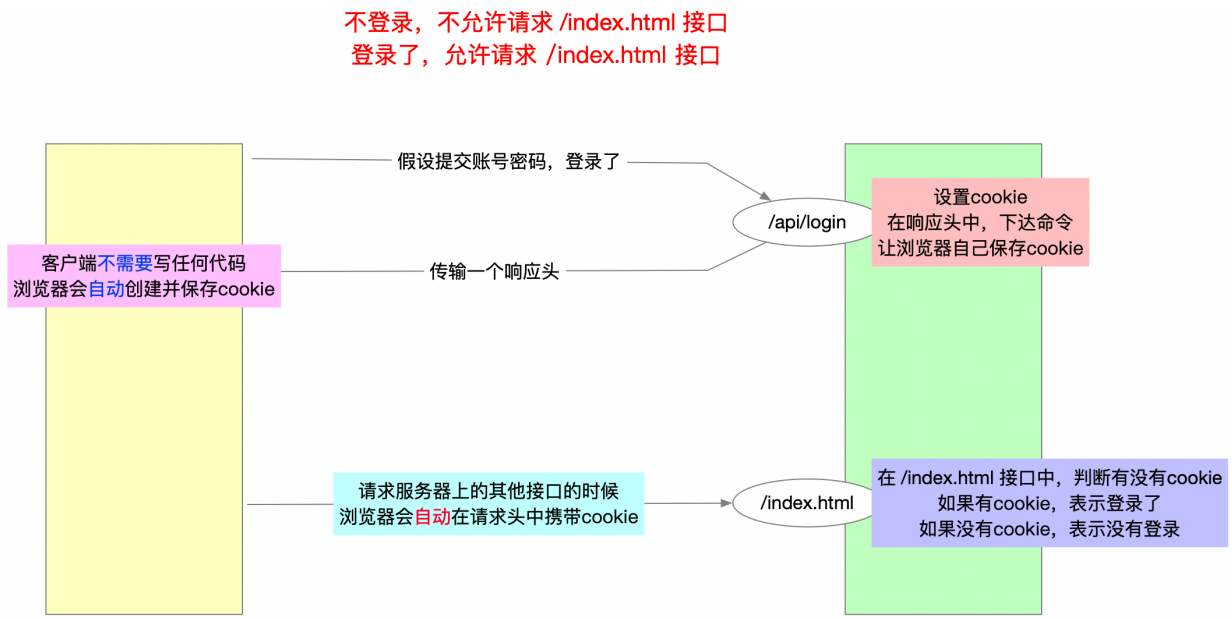
```
| - app.js              (搭建服务器)
| - public              (public文件夹用于存放前端页面)
    | - index.html      (一个前端的html页面)
```

    - app.js编写接口 `/index.html`
      - 接口中，使用fs读取文件，并替换内容，最后响应给客户端
      - 客户端请求 `http://localhost:3006/index.html`

```
// 接口，提供index.html 页面
app.get('/index.html', (req, res) => {
    // 客户端发来请求，希望看到index.html 页面。
```

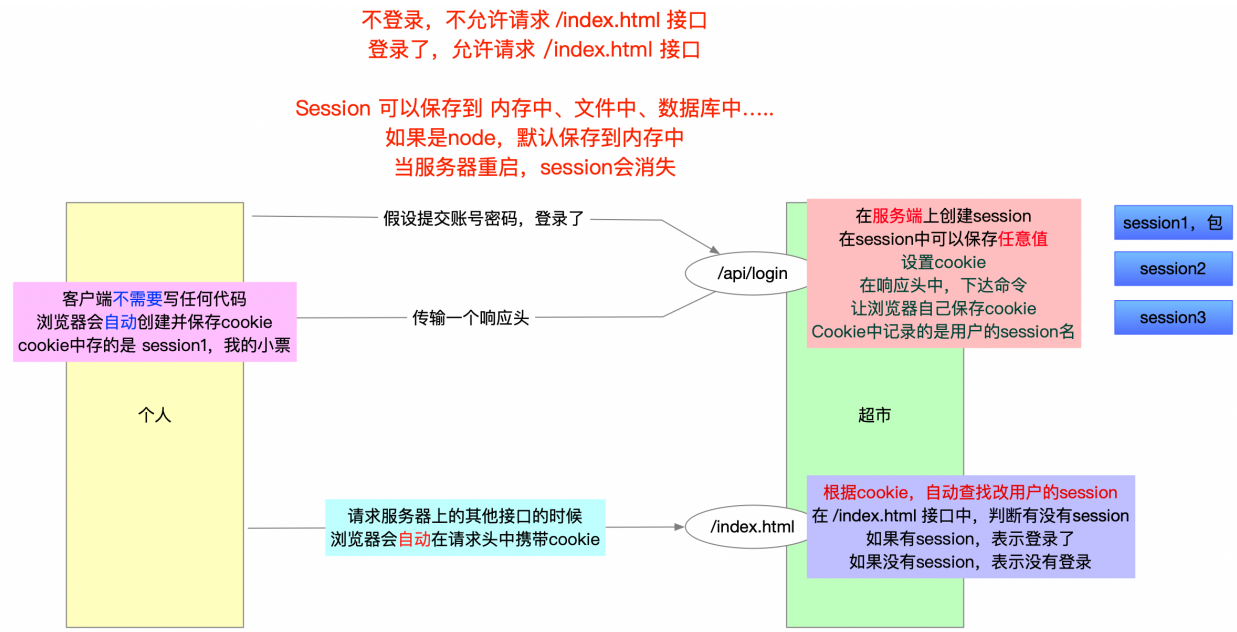
```
    // 服务器，把html页面读取出来，把读取的结果响应给客户端即可
    fs.readFile('./public/index.html', 'utf-8', (err, data) => {
        if (err) throw err;
        // console.log(data);
        // 假设从数据库中查询到了标题和内容
        data = data.replace('{{title}}', '咏鹅');
        data = data.replace('{{content}}', '鹅鹅鹅，曲项向天歌')
        res.send(data);
    })
});
```

- Cookie
  - 原理图
    - 身份认证，要完成的是：不登录，不允许访问其他页面。



  - 实现身份认证
    - 搭建基础的服务器（或者直接使用前面的传统服务端渲染模式代码）
    - 中间件配置 cookie-parser
      - app.use(cookieParser())
    - 模拟一个登录接口
      - 如果登录成功，设置cookie。`res.cookie('key', 'value', 配置项);`
    - /index.html 接口中，根据cookie判断是否登录，从而完成身份认证
  - 优缺点
    - 优点
      - 体积小
      - 客户端存放，不占用服务器空间
      - 浏览器会`自动`携带，不需要写额外的代码，比较方便
    - 缺点

        - 客户端保存，安全性较低。但可以存放加密的字符串来解决
        - 只能存字符串，cookie的大小也是有限制的
        - 可以实现跨域，但是难度大，难理解，代码难度高
        - 不适合前后端分离式的开发
    - 适用场景
        - 传统的服务器渲染模式
        - 存储安全性较低的数据，比如视频播放位置等
- Session身份认证
    - 原理图
        - 要实现的效果：不登录，不允许访问其他接口



    - 实现身份认证
        - 搭建基础的服务器
            - 下载安装第三方模块 `express` 和 `express-session`
            - 创建app.js
            - 加载所需模块
                - const express = require('express');
                - const session = require('express-session');
        - 中间件配置 session

```
app.use(session({
    secret: 'adfasdf', // 这个随便写
    saveUninitialized: false,
    resave: false
}));
```

        - 完成登录接口

        - 如果登录成功，使用session记录用户信息。

```
req.session.isLogin = true;
req.session.username = 'laotang';
```

    - /index.html 接口中，根据session判断是否登录，从而完成身份认证
- 优缺点
    - 优点
        - 服务端存放，安全性较高
        - 浏览器会自动携带cookie，不需要写额外的代码，比较方便
        - 适合服务器端渲染模式
    - 缺点
        - 会占用服务器端空间
        - session实现离不开cookie，如果浏览器禁用cookie，session不好实现
        - 不适合前后端分离式的开发
- 适用场景
    - 传统的服务器渲染模式
    - 安全性要求较高的数据可以使用session存放，比如用户私密信息、验证码等